

Module 1: Smart Roommate & House

MatchingOwner: Property & Room Listing Engine (Miftelul), Mess Court

Conflict-Resolution (Araf)Module OverviewA compatibility-driven roommate and house matching tool designed for shared living and student mess environments. The module matches potential roommates with each other and with property listings based on weighted lifestyle preferences (budget, sleep schedules, cleanliness). It manages the complete joining workflow—from pre-request discovery to post-move-in conflict logging—ensuring transparent,

race-condition-free roommate selection while protecting resident privacy.Core Features

1. Profile & Preference ArchitectureEach resident establishes a standardized Matching Profile used by the scoring engine:

Financial Limits: Budget range (min/max rent per month) and shared utility preferences.

Living Habits: Sleep schedule (e.g., Early Riser vs. Night Owl), cleanliness

threshold (1–5 scale), noise tolerance, guest/visitor policy, and smoking/pets policy.

Custom Preference Weighting: Users assign importance levels (Must-Have / High / Medium / Low) to

each factor so compatibility scoring reflects personal dealbreakers rather than flat averages.

2. Multi-Level Compatibility Scoring EngineThe system dynamically computes compatibility scores

(0–100%) across two vectors:

User ↔ User (Roommate Matching): Calculates a weighted

overlap score across lifestyle vectors, applying heavy penalties for hard dealbreakers (e.g.,

budget mismatch or conflicting pet policies).

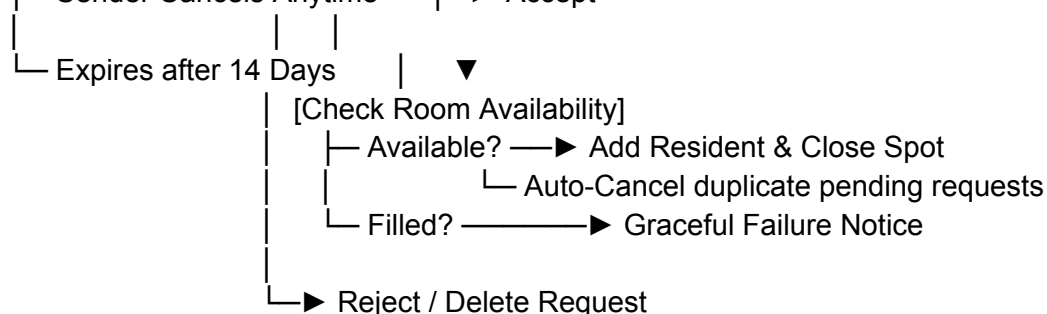
User ↔ House Listing (Group Matching): Evaluates

a user against the household aggregate average of all current residents in a multi-tenant house,

ensuring a new applicant fits the collective house culture rather than just one individual.

3. Join Request Lifecycle & Concurrency GuardrailTo handle application workflows reliably without

room double-booking:[Sender Creates Request] —> [Receiver Reviews Profile]



State Transitions: Requests move through PENDING → ACCEPTED, REJECTED,

CANCELLED, or EXPIRED.

Atomic Room Allocation: Upon accepting a join request, the system

checks real-time bed availability. If available, the applicant is locked into the house, and all other

pending requests for that specific room are instantly auto-cancelled with automated notifications

sent to remaining applicants.

4. Post-Move-In Feedback Window (Mess Court Integration)To

address profile dishonesty (e.g., exaggerating cleanliness habits):

10–15 Day Verification Window: Opens automatically on the new resident's move-in date.

Direct Dispute Routing: If a new resident violates reported habits, existing roommates can file a formal complaint during this

window.

Feedback Loop: Complaints route directly into Araf's Mess Court Conflict-Resolution

Module (Raised → Voting → Resolved). Upheld complaints apply a

penalty deduction to the offender's global match rating for future searches.

5. Data Access &

Privacy Matrix To safeguard resident personal life choices from unnecessary landlord oversight: Data Type Landlord Access Roommate/Applicant Access Contact Info & Legal Name Always Accessible Accessible after Mutual Match / Accepted Request Budget Range & Payment Status Always Accessible Visible on Match Search Lifestyle Habits (Sleep, Cleanliness) Hidden (Restricted) Visible on Match Search Dispute & Mess Court Logs Unlocked ONLY during active dispute Restricted to Household Members Scoped Enhancements In-App Pre-Request Messaging: Provides a lightweight, structured chat channel between users prior to submitting a binding join request, reducing awkward mismatches. 14-Day Automated Request Expiry: Automatically purges inactive or unresponded join requests after 14 days to keep applicant pipelines clean. Verified Profile Badge: Displays a verification badge (phone / university ID verification) on resident profiles to build trust during match discovery. Match Score Transparency Breakdown: Shows users a detailed breakdown of their compatibility score with a listing or roommate (e.g., "92% Match: High alignment on Budget & Sleep Schedule; Minor gap on Cleanliness"). Report & Block Safety System: Allows residents to block abusive users or report spam applications, immediately removing them from matching feeds.

Module 2: Daily Meal Proposal & Voting System

Owner: Mahia Tanzin

Related Modules: Shared Wallet & Bill-Splitting (Miftelul), Meal Attendance & Auto-Quantity Adjustment (Araf)

Module Overview

A democratic tool for mess management where residents propose and vote on meal plans on a day-by-day basis. Voting uses ranked-choice tallying per day to ensure fair, predictable menu selection, preventing vote-splitting and reducing daily conflicts over meal choices while keeping bulk procurement feasible.

Core Features

1. Daily Voting Mechanism

7 Parallel Daily Ballots: Instead of voting on a single rigid 7-day bundle, voting is split into 7 distinct daily ballots (Monday through Sunday) held during a single weekly window.

Ranked-Choice Tallying: Each day's winner is calculated independently using Instant Runoff Voting (IRV), preventing a dish from winning with a small minority of first-choice votes.

Multi-Tier Tie-Breaking: If ranked-choice tallying for a day ends in an exact tie after all rounds, the tie is broken by:

Proposal with higher estimated compatibility with declared dietary restrictions for that day.

Proposal with the lower estimated per-person cost.

A short, automated 6-hour daily runoff vote between the tied options.

2. Submission & Voting Schedule

Proposal Submission Window: Opens every Saturday at 6:00 PM and closes Sunday at 6:00 PM.

Main Voting Window: Opens Sunday at 6:00 PM and runs for 24 hours until Monday at 6:00 PM.

Per-Day Slotting: Proposers can submit options for specific individual days (e.g., Tuesday Lunch & Dinner) or populate a full week's worth of daily slots.

Late Submissions: Any proposal submitted after the Sunday 6:00 PM deadline is queued for the following week's cycle.

3. Per-Day Quorum & Fallback System

Quorum Threshold: A daily result is only finalized if at least 60% of active residents submit a vote for that specific day.

Isolated Extensions: If specific days fail to reach quorum while others succeed:

Reached days are locked and finalized immediately.

Unmet days receive a 12-hour voting extension with automated notifications.

Fallback Rule: If quorum is still not met after the extension, or if no proposals were submitted for a specific day, the system automatically falls back to that exact day's winning menu from the previous week (or a designated default "safe meal").

4. Proposal Editing & Lock Rules

Residents can edit or withdraw their proposed daily meals anytime before voting opens on Sunday at 6:00 PM.

Once voting begins, proposals for all days are locked. A proposer may withdraw a candidate menu, but edits are disabled to prevent mid-vote changes from invalidating cast ballots.

5. Manipulation Safeguards

One Active Proposal Per Day: Each resident may submit at most one active proposal per day of the week, preventing vote-splitting through near-duplicate options.

Account-Enforced Ballots: Each resident receives exactly one ballot per day tied to their account ID.

6. Emergency Daily Re-Vote Procedure

If a key ingredient for an upcoming winning day turns out to be unavailable:

The cook/house manager flags the item missing.

The system triggers a 3-hour fast re-vote strictly for that affected day, using only the runner-up proposals from that day's original ballot.

If the re-vote fails or time expires, that specific day falls back to the default "safe meal."

Scoped Enhancements

Variety Enforcement: Tracks dish recency. If Biryani wins Monday's vote, Biryani proposals submitted for Tuesday through Thursday receive an automatic penalty score in recommendations to discourage repetition.

Fair Voting Weight: Decays a resident's voting influence slightly across subsequent daily ballots if their proposed menus have won multiple days in recent cycles.

Dietary & Allergy Filter: Automatically flags and hides proposals that conflict with active residents' declared dietary restrictions for that day.

Live Cost & Nutrition Indicator: Displays per-person estimated cost alongside a basic nutritional snapshot (e.g., Protein-Heavy, Balanced, Light) for each daily proposal.

Blind Voting: Keeps proposer identities anonymous across all 7 daily ballots until voting closes.

Post-Meal Satisfaction Rating: Lets residents submit a quick 1–5 star rating after dinner each night, feeding feedback into the recommendation engine for future weeks.

Reusable Menu Templates: Allows residents to save high-rated daily plans as templates to re-propose in future weeks with one click.

Module 3 — Google Maps API Integration — in the same depth as Module 4's explanation.

What the module does

The core idea: during property discovery, tenants see listings plotted as pins on an embedded Google Map instead of just a text address. This is meant to help them quickly judge two things — how far a listing is from somewhere that matters to them (like work) and what the surrounding area is like — before they invest time contacting the landlord or scheduling a viewing.

Why the original description wasn't enough

The original was, again, a description of the experience the feature should create, not a specification of how it should behave. It said the map "helps tenants calculate commute distances," but calculating something and displaying a pin on a map are two different technical capabilities — one needs the Distance Matrix API and an origin input from the user, the other just needs the Maps JavaScript API and coordinates. The original sentence doesn't distinguish between them, so it's genuinely ambiguous whether "commute calculation" was ever meant to be built, or whether the intent was just "tenant looks at the map and estimates it themselves."

It also had no answer for the ordinary things that go wrong in a real system: a listing with a bad or missing address, the Maps API failing to load, or a discovery page with 200 listings all trying to render as individual pins at once. And it said nothing about cost or security, even though Google Maps APIs are metered and an exposed, unrestricted API key is a common real-world security mistake.

How the loopholes were closed

"Real-world locations" and geocoding failure were made concrete: addresses are geocoded and cached when a listing is created, and if geocoding fails, the listing simply doesn't get a map marker — it still shows up in the regular list view rather than breaking the page or silently disappearing.

"Calculate commute distances" was disambiguated into an actual requirement: the tenant enters an origin address, and the system calls the Distance Matrix API to return real distance/time for driving and transit. This turns a vague promise into something a developer can actually build and a tester can actually verify.

Marker overload was addressed with clustering — once the number of visible listings crosses a threshold, markers group together instead of turning into an unreadable pile of pins.

API failure handling was added: if the Maps API itself doesn't load (network issue, service outage), the page falls back to a list-only view instead of the entire discovery page breaking.

Privacy was addressed by only showing an approximate, street-block-level location publicly, and revealing the exact address only once a tenant has an active inquiry — a standard practice on real rental platforms to prevent exact addresses from being scraped or misused before there's a legitimate reason to share them.

Cost and security were addressed with two non-functional requirements: caching geocoding/distance results so the same listing or route isn't re-queried unnecessarily (controls API cost), and restricting the client-side API key by referrer and by which specific APIs it can call (prevents the key from being scraped out of page source and abused elsewhere).

Testability was added the same way as Module 4 — every requirement has a matching acceptance criterion, so there's a concrete definition of "this feature works" rather than a subjective one.

The additional features, and why they matter

Neighborhood data overlay: The original spec's "evaluate the surrounding area" was essentially an unbacked promise — the map alone doesn't tell you anything about a neighborhood beyond its shape. Pulling in real data (walk score, nearby transit, school ratings) is what actually delivers on that original claim instead of leaving it as marketing language with nothing behind it.

Multi-origin commute comparison: Most real house-hunting isn't done by one person with one commute to worry about — couples and roommates often need a listing that works reasonably for two different destinations. Letting a tenant compare a listing against two saved origins makes the commute feature useful for how people actually search, not just a single idealized user.

Commute-based filtering/sorting: Showing a commute time after the fact is useful, but letting a tenant filter listings by "under 30 minutes from work" upfront is a much stronger use of the same underlying data — it turns commute from a passive display into an actual search tool.

Saved search radius alerts: This extends the feature's value past the initial discovery session — instead of a tenant manually re-checking the map, the system proactively tells them when something new matches their commute preference.

Street View preview: A small but meaningful addition — a tenant can get a feel for the physical block (is it a busy road, a quiet residential street, etc.) without leaving the discovery flow, which supports the "evaluate the surrounding area" goal in a very concrete, low-effort way.

Module 4 — Automated Chore Rotation

— covering what it does, why the original version had gaps, how those were fixed, and how the new features extend it.

What the module does

The core idea: residents in a household get weekly chores (cleaning, grocery runs, etc.) automatically assigned and rotated so no one gets stuck with the same task every week, and each assignment is pushed straight into the resident's personal Google Tasks with a due date and reminder — so people don't have to check a separate app to know what they owe the household that week.

Why the original description wasn't enough

As written, it was a one-sentence idea, not a spec. It said what the feature should feel like to a user, but not how it should behave in the situations that actually come up in a real household:

How exactly does rotation work, and is it fair over time?

What happens when someone's away, hasn't connected their Google account, or the API push fails?

Does it just dump chores into a resident's existing task list, or keep them separate?
Is there any way to know a chore actually got done, or is it fire-and-forget?

None of that was answered — which means two people building this from the same one-liner could build two very different, possibly broken systems.

How the loopholes were closed

Rotation logic was made concrete: a defined round-robin cycle, with an explicit rule for what happens when chores don't divide evenly among residents, so "doubling up" itself rotates instead of always landing on the same person.

Absence handling was added: a resident can mark themselves unavailable for a week, and rotation skips them without permanently breaking the underlying order — so being on vacation once doesn't throw off fairness long-term.

Manual override was added: residents can propose and confirm a direct swap, so the system isn't rigidly automatic with no human flexibility.

Google account dependency was addressed on two fronts: what happens if a resident never connects their account (flagged + admin notified, not silently skipped), and what happens if a previously-connected account's access expires or is revoked (retry, then notify through a backup channel like email).

API failure handling was added: if the push to Google Tasks fails (rate limit, outage), the system retries rather than just losing the assignment.

List separation was specified: chores go into a dedicated "Household Chores" list in Google Tasks rather than merging into a resident's personal to-do list, avoiding clutter and confusion about which tasks are "chore system" vs. personal.

Stale task handling was added: if a rotation changes after a task was already pushed (a swap happens, someone goes unavailable), the old pushed task gets updated or removed instead of leaving a duplicate sitting in the resident's account.

Privacy/permission scope was addressed: the system requests only the minimum OAuth access needed and discloses to residents what it can read/write before they connect their account — since writing to someone's personal Google account is a meaningful trust boundary.

Completion tracking was added: the system reads completion status back from Google Tasks, so the household has actual visibility into whether chores got done, not just that they got assigned.

Testability was added throughout: every requirement above has a matching acceptance criterion, so "is this feature working correctly" has a concrete answer instead of being a judgment call.

The five features you chose to add, and why they matter

These extend the module beyond the core spec into things that solve real household friction points, without changing the underlying rotation engine:

Chore marketplace (swap beyond 1-to-1): Direct swaps only work if you find someone willing to trade at the same time you are. A shared pool where anyone can claim a posted chore is much more realistic for how roommates actually negotiate — "I'll take dishes off anyone's hands this week" is a common real pattern that rigid pairwise swapping doesn't support.

Smart rescheduling around calendar conflicts: Fairness in rotation doesn't help if someone's due date lands during finals week or a work trip. Reading calendar availability (with explicit, separate consent) lets the system proactively flag a bad due date and suggest an alternative before it's missed, rather than the household finding out only after a chore didn't happen.

Recurring chore auto-detection / smarter due dates: People have natural patterns — someone might always do grocery runs Friday, not Sunday. Learning that and suggesting a better default due date (only with explicit resident confirmation, never silently) makes the system feel less like a rigid scheduler and more like it's actually adapting to the household.

Chore splitting for large tasks: Not every chore is equal-sized. "Deep clean the kitchen" isn't fairly comparable to "take out trash." Letting a large chore be broken into sub-tasks assigned across multiple residents in the same week — with the parent task only completing once all sub-tasks do — makes big jobs shareable instead of one person absorbing the whole load.

Completion quality feedback: Rotation and completion-tracking answer "did it happen," but not "was it actually done well." An optional, anonymous, household-toggable rating system gives a low-friction way to surface "technically done but not really" without turning it into a direct confrontation between roommates — and it's opt-in specifically because not every household wants this level of scrutiny.